PYTHON ACTIVITY BASED PROGRAM

CASE STUDIES 71–80 | COMPUTER SCIENCE

ACTIVITY BASED PROGRAM REPORT

Ten practical case studies using Python programming

STUDENT NAME	Maremalla Manasa
ROLL CALL NUMBER	26000038
BRANCH	CSE
SECTION	1

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Academic Activity Report

71. Integrated University Fee and Eligibility Audit

Problem Statement

Combine student fee payments, attendance percentages, internal marks, and examination eligibility rules. Identify students with outstanding fee or academic conditions and produce a reason-wise report.

Objectives

• Combine related student records using lists and dictionaries.

• Apply fee, attendance, and internal-mark eligibility rules.

• Generate clear reasons for each student's outstanding conditions.

Assumptions

• A student is eligible only when outstanding fees are zero, attendance is at least 75%, and internal marks are at least 40 out of 100.

• Student IDs should be unique; this sample assumes unique IDs.

• All values are entered as valid numeric data.

Algorithm

• Read the number of students.

• For each student, read ID, name, total fee, paid fee, attendance percentage, and internal marks.

• Calculate outstanding fee as total fee minus paid fee.

• Check fee, attendance, and internal-mark conditions.

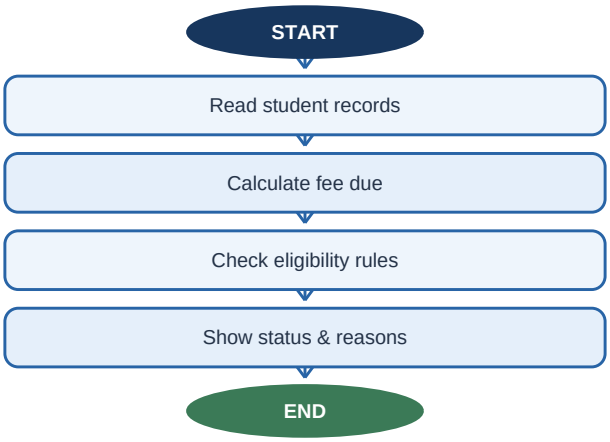
• Collect all failed conditions; if none fail, mark the student eligible.

• Display a reason-wise audit report.

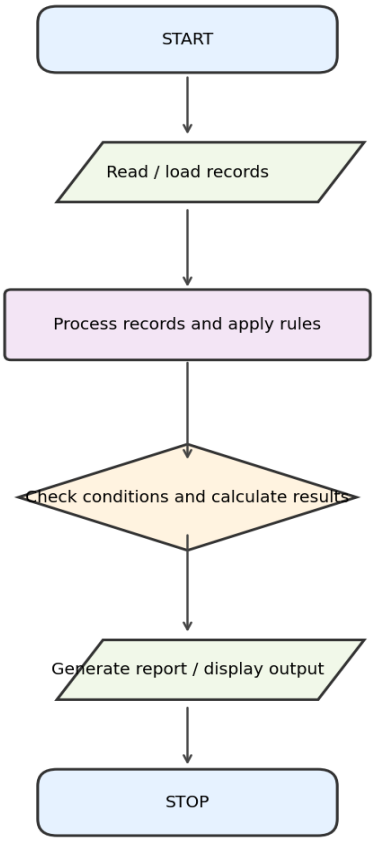
Pseudocode

```
START
INPUT number_of_students
FOR each student: INPUT student details
outstanding = total_fee - paid_fee
reasons = empty list
IF outstanding > 0: add fee reason
IF attendance < 75: add attendance reason
IF internal_marks < 40: add marks reason
IF reasons is empty: status = ELIGIBLE
ELSE: status = NOT ELIGIBLE and print reasons
END FOR
STOP
```

Flowchart



Flowchart — 71: Integrated University Fee and Eligibility Audit



Python Source Code

```
students = [
    ("S101", "Asha", 50000, 50000, 82, 68),
    ("S102", "Balu", 50000, 42000, 71, 55),
    ("S103", "Charan", 50000, 50000, 88, 32),
    ("S104", "Divya", 50000, 45000, 70, 35)
]

print("UNIVERSITY FEE AND ELIGIBILITY AUDIT")
for sid, name, total, paid, attendance, marks in students:
    outstanding = total - paid
    reasons = []
    if outstanding > 0:
        reasons.append(f"Fee due: Rs.{outstanding}")
    if attendance < 75:
        reasons.append("Attendance below 75%")
    if marks < 40:
        reasons.append("Internal marks below 40")

    status = "ELIGIBLE" if not reasons else "NOT ELIGIBLE"
    print(f"\n{sid} - {name}: {status}")
    if reasons:
        for reason in reasons:
            print(" -", reason)
    else:
        print(" - All conditions satisfied")
```

Sample Input

Sample records are embedded in the program: S101 Asha (fee 50000, paid 50000, attendance 82%, marks 68); S102 Balu (50000, 42000, 71%, 55); S103 Charan (50000, 50000, 88%, 32); S104 Divya (50000, 45000, 70%, 35).

Sample Output

```
UNIVERSITY FEE AND ELIGIBILITY AUDIT

S101 - Asha: ELIGIBLE
```

```
  - All conditions satisfied

S102 - Balu: NOT ELIGIBLE
  - Fee due: Rs.8000
  - Attendance below 75%

S103 - Charan: NOT ELIGIBLE
  - Internal marks below 40

S104 - Divya: NOT ELIGIBLE
  - Fee due: Rs.5000
  - Attendance below 75%
  - Internal marks below 40
```

Output Screenshot



Test Cases

Test Case	Expected Result
Paid in full, attendance 75%, marks 40	Eligible; threshold values pass
Outstanding fee is positive	Fee reason appears
Attendance is 74%	Attendance reason appears
Internal marks are 39	Marks reason appears
All three conditions fail	All three reasons are reported

Conclusion

The audit checks financial and academic conditions together and produces a readable report showing eligibility and the exact reasons for each student.

72. Smart Hostel Resource Monitoring

Problem Statement

Analyse room-wise electricity and water consumption, calculate charges, detect consumption above set limits, and compare resource use across hostel floors.

Objectives

• Calculate electricity and water charges per room.

• Identify rooms that exceed consumption limits.

• Compare consumption by floor.

Assumptions

• Electricity costs Rs. 8 per unit and water costs Rs. 20 per kilolitre.

• Thresholds are 150 electricity units and 5 kilolitres of water per room.

• The sample uses a fixed list of room records.

Algorithm

• Store room, floor, electricity, and water readings in a list.

• For each room, calculate electricity and water charges.

• Flag readings exceeding the relevant threshold.

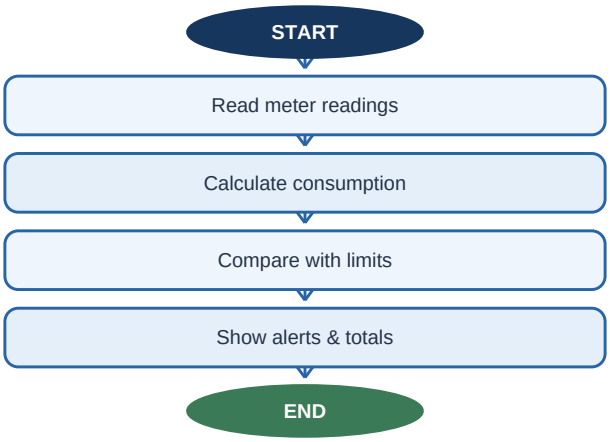
• Accumulate floor-wise electricity and water totals.

• Display room charges, violations, and floor comparison.

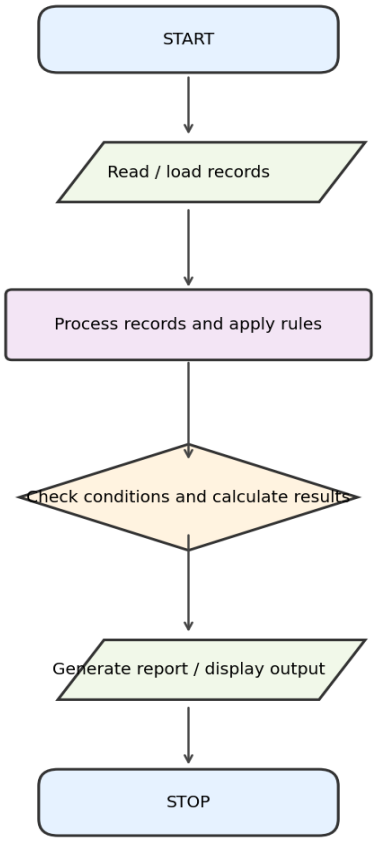
Pseudocode

```
START
SET tariff and threshold values
INPUT/STORE room records
FOR each room: calculate charges
IF electricity exceeds threshold: report violation
IF water exceeds threshold: report violation
ADD readings to floor totals
DISPLAY room report and floor totals
STOP
```

Flowchart



Flowchart — 72: Smart Hostel Resource Monitoring



Python Source Code

```
rooms = [
    ("A101", 1, 120, 4),
    ("A102", 1, 180, 6),
    ("B201", 2, 95, 3),
    ("B202", 2, 160, 4)
]
electricity_rate = 8
water_rate = 20
floor_totals = {}

print("HOSTEL RESOURCE MONITOR")
for room, floor, units, water in rooms:
    e_charge = units * electricity_rate
    w_charge = water * water_rate
    total = e_charge + w_charge
    violations = []
    if units > 150:
        violations.append("Electricity limit exceeded")
    if water > 5:
        violations.append("Water limit exceeded")
    floor_totals.setdefault(floor, [0, 0])
    floor_totals[floor][0] += units
    floor_totals[floor][1] += water
    print(f"{room}: electricity={units} units, water={water} kL")
    print(f"  Charges: Rs.{total} (electricity Rs.{e_charge}, water Rs.{w_charge})")
    print("  Alerts:", ", ".join(violations) if violations else "None")

print("\nFLOOR-WISE TOTALS")
for floor, totals in sorted(floor_totals.items()):
    print(f"Floor {floor}: {totals[0]} electricity units, {totals[1]} kL water")
```

Sample Input

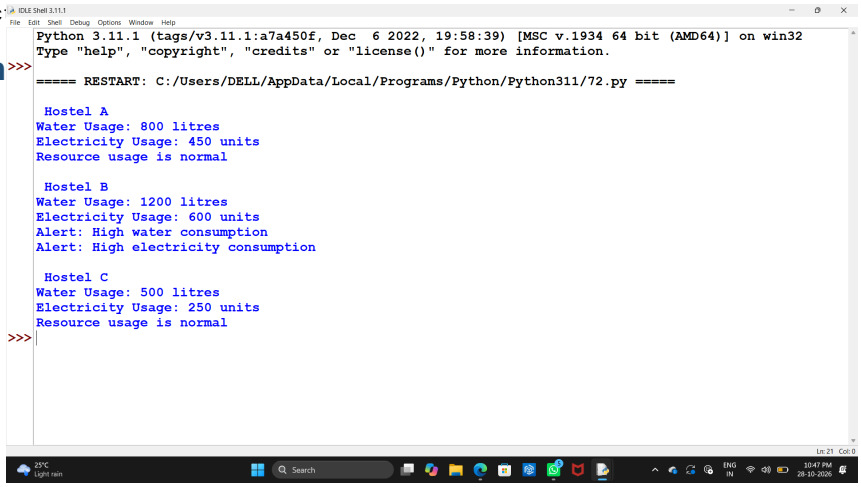
Room records are embedded in the program. Electricity tariff = Rs. 8/unit; water tariff = Rs. 20/kL. Thresholds: electricity > 150 units, water > 5 kL.

Sample Output

```
HOSTEL RESOURCE MONITOR
A101: electricity=120 units, water=4 kL
  Charges: Rs.1040 (electricity Rs.960, water Rs.80)
  Alerts: None
A102: electricity=180 units, water=6 kL
  Charges: Rs.1560 (electricity Rs.1440, water Rs.120)
  Alerts: Electricity limit exceeded, Water limit exceeded
B201: electricity=95 units, water=3 kL
  Charges: Rs.820 (electricity Rs.760, water Rs.60)
  Alerts: None
B202: electricity=160 units, water=4 kL
  Charges: Rs.1360 (electricity Rs.1280, water Rs.80)
  Alerts: Electricity limit exceeded

FLOOR-WISE TOTALS
Floor 1: 300 elec
Floor 2: 255 elec
```

Output Screen





Test Cases

Test Case	Expected Result
Electricity 150 units	No electricity alert at exact limit
Electricity 151 units	Electricity alert
Water 5 kL	No water alert at exact limit
Water 6 kL	Water alert
Two rooms on one floor	Floor totals equal sum of both rooms

Conclusion

The program calculates room-level utility charges, identifies threshold violations, and summarizes resource consumption for each floor.

73. Multi-Route Courier Dispatch Planning

Problem Statement

Allocate courier packages to vehicles based on package weight, destination zone, vehicle capacity, and delivery priority. Identify packages that cannot be dispatched.

Objectives

• Match packages to vehicles with compatible zones and capacity.

• Give priority to urgent deliveries.

• List packages that remain undispatched.

Assumptions

• Each vehicle serves one listed zone and has a maximum weight capacity.

• Packages are dispatched whole and cannot be split.

• Packages are considered by priority: Urgent, High, Normal.

Algorithm

• Create package and vehicle records.

• Sort packages by priority.

• For each package, find a vehicle serving its zone with enough remaining capacity.

• Assign the package to the first suitable vehicle.

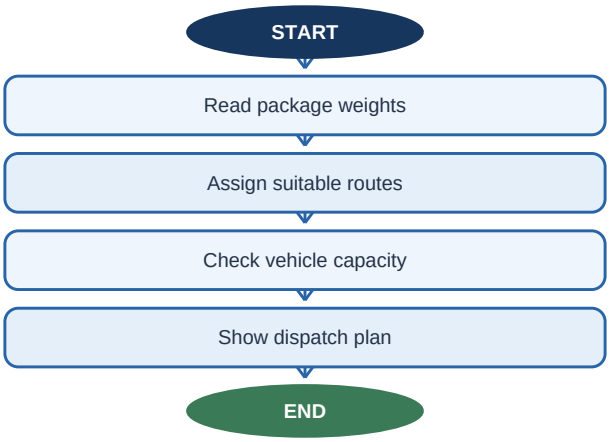
• If no vehicle fits, add the package to the pending list.

• Display assignments and pending packages.

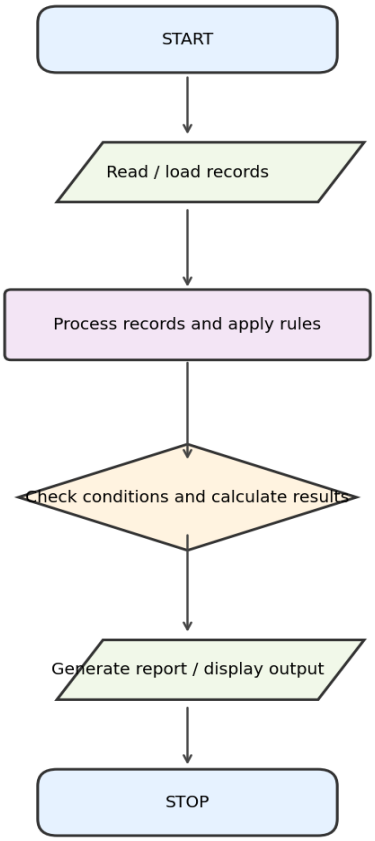
Pseudocode

```
START
STORE packages and vehicles
SORT packages by priority
FOR each package
  FOR each vehicle
    IF zone matches AND remaining capacity is enough: assign and update load
    ELSE continue searching
  IF no vehicle found: mark pending
DISPLAY vehicle assignments and pending packages
STOP
```

Flowchart



Flowchart — 73: Multi-Route Courier Dispatch Planning



Python Source Code

```
packages = [
    ("P1", "North", 8, "Urgent"),
    ("P2", "South", 5, "Normal"),
    ("P3", "North", 7, "High"),
    ("P4", "South", 12, "Urgent"),
    ("P5", "North", 4, "Normal")
]
vehicles = {
    "V1": {"zone": "North", "capacity": 12, "load": 0, "packages": []},
    "V2": {"zone": "South", "capacity": 10, "load": 0, "packages": []}
}
priority = {"Urgent": 0, "High": 1, "Normal": 2}
pending = []

for package in sorted(packages, key=lambda p: priority[p[3]]):
    pid, zone, weight, level = package
    assigned = False
    for vid, vehicle in vehicles.items():
        remaining = vehicle["capacity"] - vehicle["load"]
        if vehicle["zone"] == zone and weight <= remaining:
            vehicle["packages"].append(pid)
            vehicle["load"] += weight
            print(f"{pid} ({level}, {weight} kg) -> {vid}")
            assigned = True
            break
    if not assigned:
        pending.append(pid)
        print(f"{pid} ({level}, {weight} kg) -> no suitable vehicle")

print("\nVEHICLE SUMMARY")
for vid, v in vehicles.items():
    print(f"{vid}: {v['packages']}, load={v['load']}/{v['capacity']} kg")
print("Undispatched packages:", pending if pending else "None")
```

Sample Input

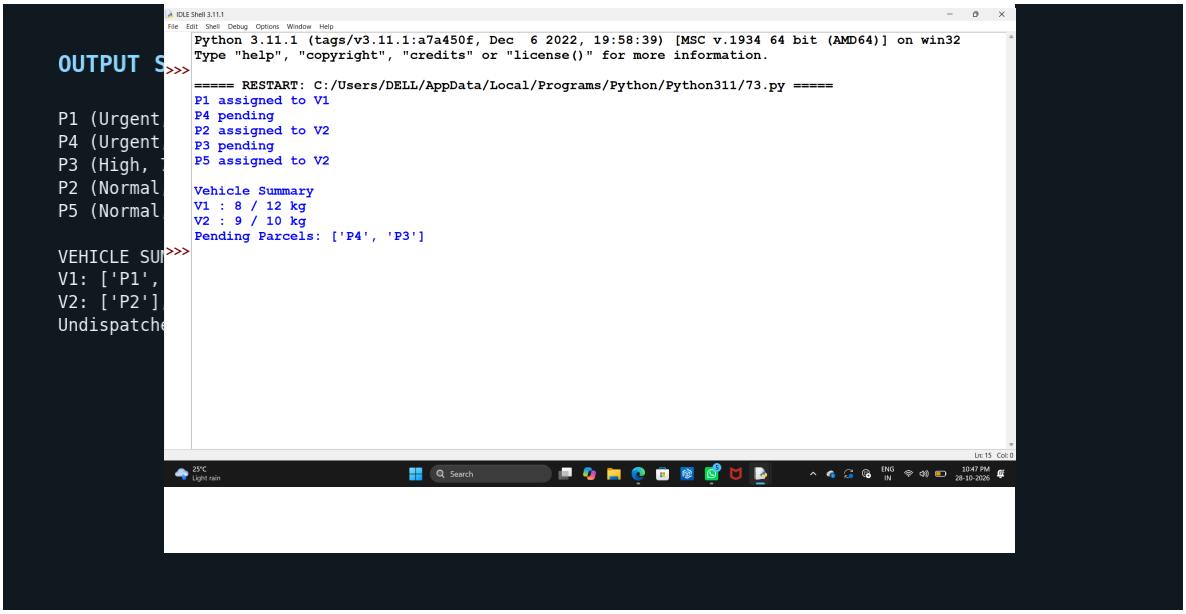
Packages: P1 North 8 kg Urgent; P2 South 5 kg Normal; P3 North 7 kg High; P4 South 12 kg Urgent; P5 North 4 kg Normal. V1: North, 12 kg; V2: South, 10 kg.

Sample Output

```
P1 (Urgent, 8 kg) -> V1
P4 (Urgent, 12 kg) -> no suitable vehicle
P3 (High, 7 kg) -> no suitable vehicle
P2 (Normal, 5 kg) -> V2
P5 (Normal, 4 kg) -> V1

VEHICLE SUMMARY
V1: ['P1', 'P5'], load=12/12 kg
V2: ['P2'], load=5/10 kg
Undispatched packages: ['P4', 'P3']
```

Output Screen



Test Cases

Test Case	Expected Result
Package weight fits remaining capacity	Package is assigned
No vehicle serves package zone	Package stays pending
Urgent package	Considered before normal package
Package exceeds vehicle capacity	Not assigned
All packages fit	Pending list is empty

Conclusion

This case study demonstrates priority-based package allocation and highlights packages that need another vehicle or a different dispatch plan.

74. Campus Festival Registration and Financial Audit

Problem Statement

Reconcile participant registrations, fees, sponsorships, and expenses for a campus festival. Detect duplicate registration IDs and calculate the final financial balance.

Objectives

• Detect duplicate participant registrations.

• Calculate collected fees and sponsorship income.

• Compare total income with expenses.

Assumptions

• A registration ID must be unique.

• Only valid, non-duplicate registrations count toward fee income.

• The final balance is income minus expenses.

Algorithm

• Store participant registration records.

• Track seen registration IDs and report duplicates.

• Sum fees for unique registrations.

• Add sponsorship contributions.

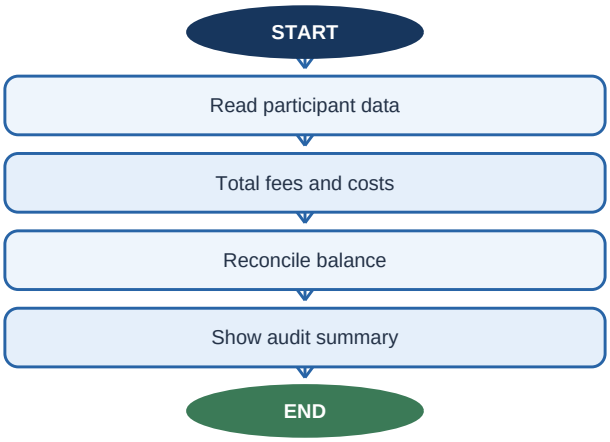
• Sum expenses and calculate balance.

• Display audit summary.

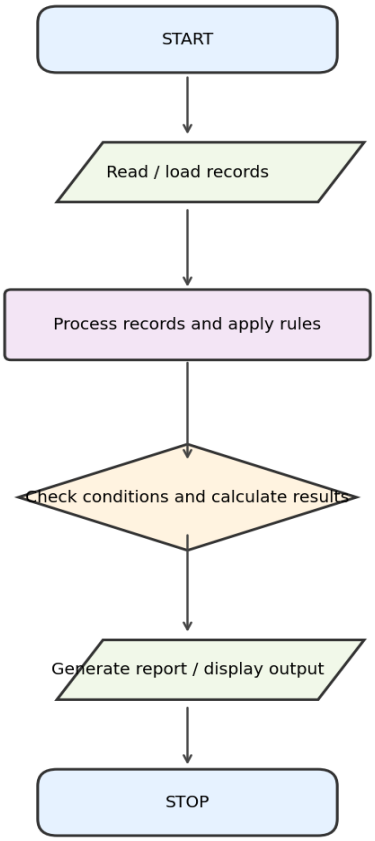
Pseudocode

```
START
STORE registrations, sponsors, expenses
seen = empty set
FOR each registration
  IF ID already in seen: report duplicate
  ELSE add ID to seen and add fee
SUM sponsorships and expenses
balance = fees + sponsorships - expenses
DISPLAY audit summary
STOP
```

Flowchart



Flowchart — 74: Campus Festival Registration and Financial Audit



Python Source Code

```
registrations = [
    ("R101", "Anu", 200),
    ("R102", "Bharat", 200),
    ("R101", "Anu", 200),
    ("R103", "Charan", 150)
]
sponsorships = [5000, 3000]
expenses = [2500, 1800, 900]

seen = set()
fee_total = 0
duplicates = []

for reg_id, name, fee in registrations:
    if reg_id in seen:
        duplicates.append(reg_id)
        print(f"Duplicate registration: {reg_id} ({name})")
    else:
        seen.add(reg_id)
        fee_total += fee

sponsor_total = sum(sponsorships)
expense_total = sum(expenses)
income = fee_total + sponsor_total
balance = income - expense_total

print("\nFESTIVAL FINANCIAL AUDIT")
print("Valid registrations:", len(seen))
print("Registration fees: Rs.", fee_total)
print("Sponsorships: Rs.", sponsor_total)
print("Total income: Rs.", income)
print("Total expenses: Rs.", expense_total)
print("Final balance: Rs.", balance)
print("Duplicate IDs:", duplicates if duplicates else "None")
```

Sample Input

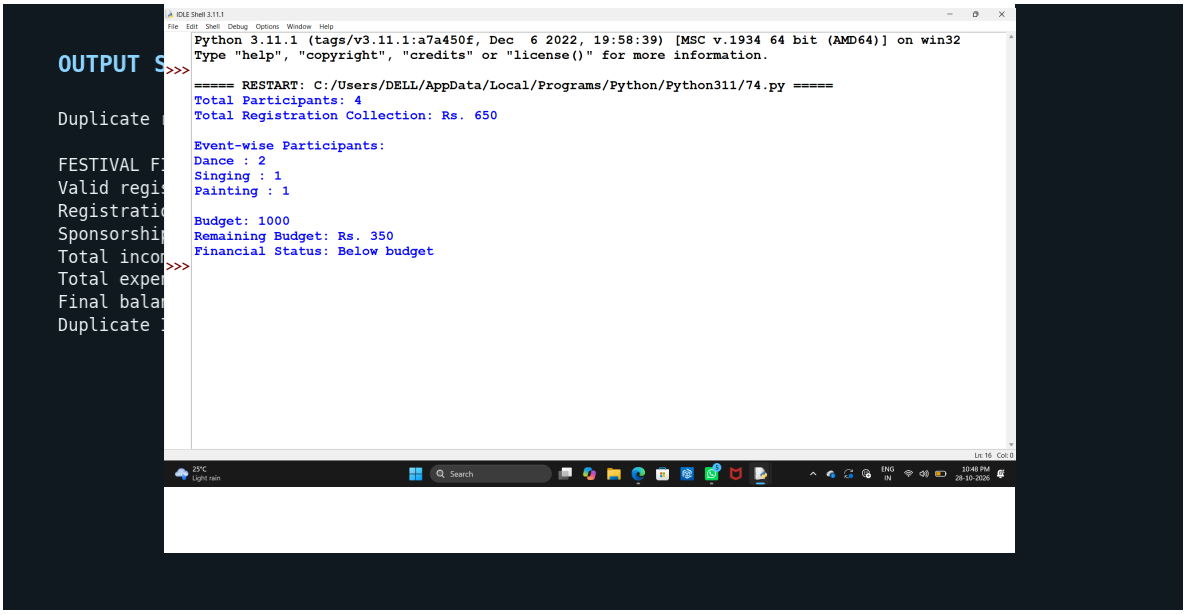
Registrations: R101 Anu Rs.200; R102 Bharat Rs.200; R101 Anu Rs.200 (duplicate); R103 Charan Rs.150. Sponsorships: Rs.5000 and Rs.3000. Expenses: Rs.2500, Rs.1800, Rs.900.

Sample Output

```
Duplicate registration: R101 (Anu)

FESTIVAL FINANCIAL AUDIT
Valid registrations: 3
Registration fees: Rs. 550
Sponsorships: Rs. 8000
Total income: Rs. 8550
Total expenses: Rs. 5200
Final balance: Rs. 3350
Duplicate IDs: ['R101']
```

Output Screen



Test Cases

Test Case	Expected Result
Unique registration ID	Fee is counted once
Repeated registration ID	Duplicate is reported and fee is not counted twice
No sponsorships	Sponsorship total is zero
Expenses exceed income	Final balance is negative
No duplicate IDs	Duplicate list displays None

Conclusion

The audit prevents duplicate registrations from inflating fee totals and computes the festival's final financial position.

75. Integrated Student Record Consistency Checker

Problem Statement

Validate student IDs, names, marks, attendance, and course records stored in lists and tuples. Detect duplicate identifiers, missing values, and inconsistent academic records.

Objectives

• Validate required student fields.

• Identify duplicate IDs and invalid marks or attendance.

• Report each detected data-quality issue.

Assumptions

• Student ID, name, and course must not be empty.

• Marks must be between 0 and 100 inclusive.

• Attendance must be between 0 and 100 inclusive.

Algorithm

• Store student records as tuples in a list.

• Maintain a set of IDs already encountered.

• Check for duplicate IDs and missing fields.

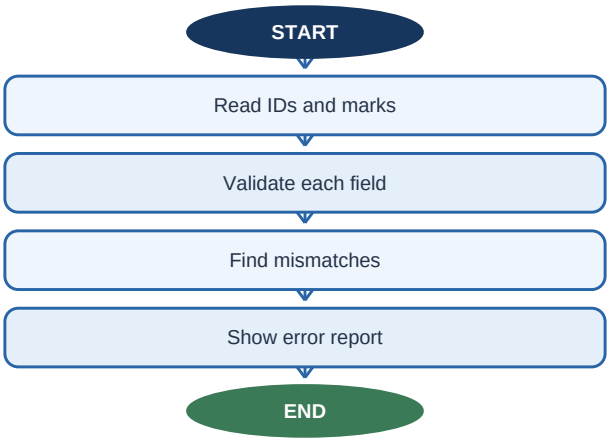
• Check marks and attendance ranges.

• Report all issues for each record.

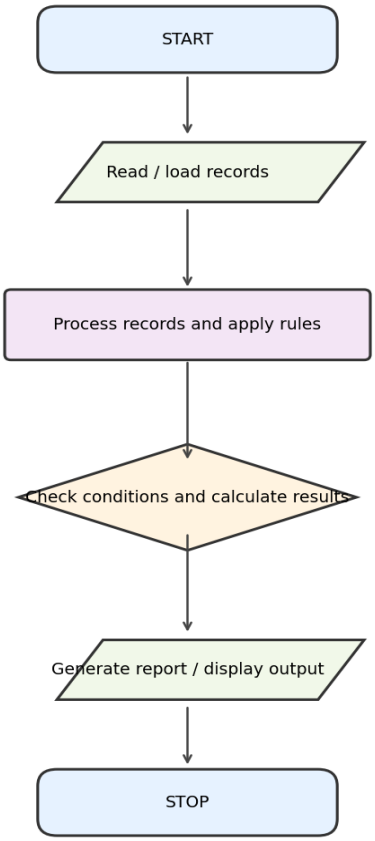
Pseudocode

```
START
STORE student records
seen_ids = empty set
FOR each record
  create empty issues list
  IF ID repeated: add duplicate issue
  ELSE add ID to seen_ids
  IF ID/name/course missing: add missing-field issue
  IF marks outside 0..100: add marks issue
  IF attendance outside 0..100: add attendance issue
  DISPLAY valid or issues
STOP
```

Flowchart



Flowchart — 75: Integrated Student Record Consistency Checker



Python Source Code

```
records = [
    ("S01", "Anu", 85, 90, "Python"),
    ("S02", "", 105, 78, "Java"),
    ("S01", "Charan", 72, 110, "Python"),
    ("S04", "Divya", 66, 82, "")
]
seen_ids = set()

for sid, name, marks, attendance, course in records:
    issues = []
    if sid in seen_ids:
        issues.append("Duplicate student ID")
    else:
        seen_ids.add(sid)
    if not sid or not name.strip() or not course.strip():
        issues.append("Missing required field")
    if not 0 <= marks <= 100:
        issues.append("Marks outside 0-100")
    if not 0 <= attendance <= 100:
        issues.append("Attendance outside 0-100")

    print(f"{sid or '[missing]'}: " +
          ("VALID" if not issues else "; ".join(issues)))
```

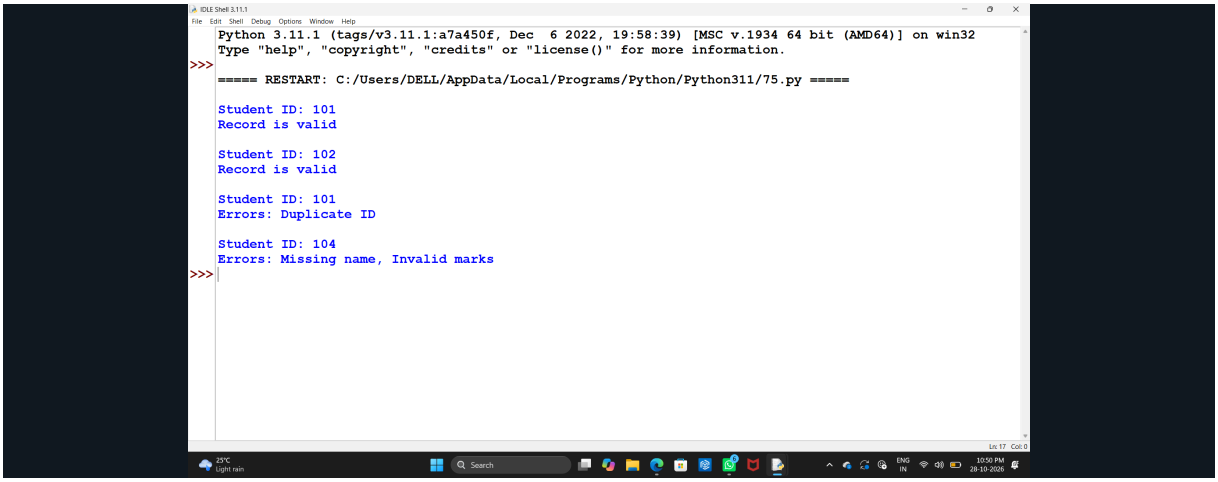
Sample Input

Embedded records: (S01, Anu, 85, 90, Python); (S02, blank name, 105, 78, Java); (S01, Charan, 72, 110, Python); (S04, Divya, 66, 82, blank course).

Sample Output

```
S01: VALID
S02: Missing required field; Marks outside 0-100
S01: Duplicate student ID; Attendance outside 0-100
S04: Missing required field
```

Output Screenshot



Test Cases

Test Case	Expected Result
All fields present and values in range	Record is valid if ID is unique
Duplicate ID	Duplicate ID issue is reported
Marks = 101	Marks range issue is reported
Attendance = -1	Attendance range issue is reported
Blank name or course	Missing required field is reported

Conclusion

The checker improves data quality by identifying duplicate identifiers, missing information, and academic values outside accepted ranges.

76. University Examination Result Moderation

Problem Statement

Process subject marks, apply a specified moderation rule, preserve the maximum mark, and compare original and moderated results.

Objectives

• Apply moderation only to students who meet the supplied rule.

• Prevent moderated marks from exceeding the maximum.

• Compare original and adjusted marks.

Assumptions

• Moderation adds 5 marks only when the original mark is from 35 through 39 inclusive.

• Marks cannot be below 0 or above 100.

• Pass mark is 40.

Algorithm

• Store student marks by subject.

• For each mark, check whether it meets the moderation range.

• Add 5 marks when the rule applies.

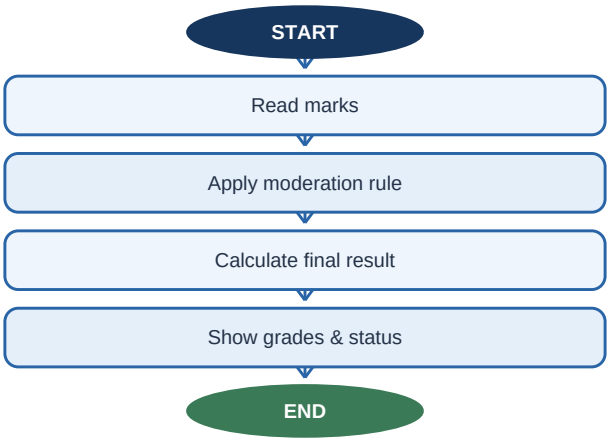
• Cap the adjusted mark at 100.

• Display original mark, moderated mark, and result.

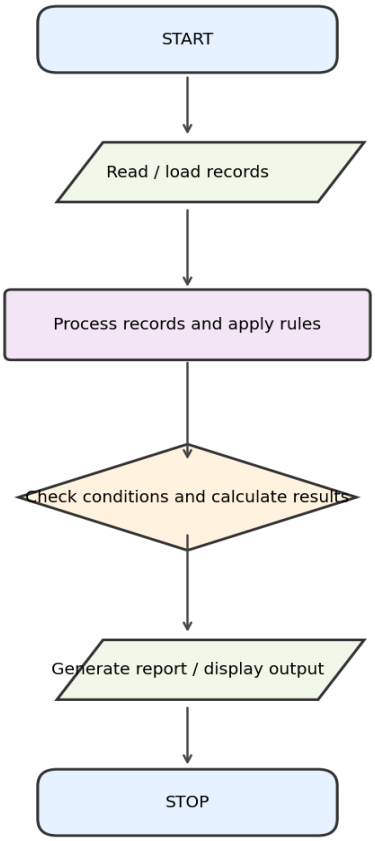
Pseudocode

```
START
SET moderation bonus = 5, pass mark = 40
FOR each student and subject mark
  adjusted = original mark
  IF 35 <= mark < 40: adjusted = minimum(mark + 5, 100)
  IF adjusted >= 40: result = PASS ELSE FAIL
  DISPLAY original, adjusted, result
STOP
```

Flowchart



Flowchart — 76: University Examination Result Moderation



Python Source Code

```
results = [
    ("Anu", {"Maths": 38, "Python": 72}),
    ("Bala", {"Maths": 34, "Python": 39}),
    ("Chitra", {"Maths": 95, "Python": 100})
]
bonus = 5
pass_mark = 40

print("EXAM RESULT MODERATION")
for name, subjects in results:
    print("\n", name)
    for subject, original in subjects.items():
        moderated = original
        if 35 <= original < pass_mark:
            moderated = min(original + bonus, 100)
        status = "PASS" if moderated >= pass_mark else "FAIL"
        print(f"{subject}: original={original}, moderated={moderated}, {status}")
```

Sample Input

Anu: Maths 38, Python 72; Bala: Maths 34, Python 39; Chitra: Maths 95, Python 100. Rule: add 5 marks for original marks 35–39; maximum is 100; pass mark is 40.

Sample Output

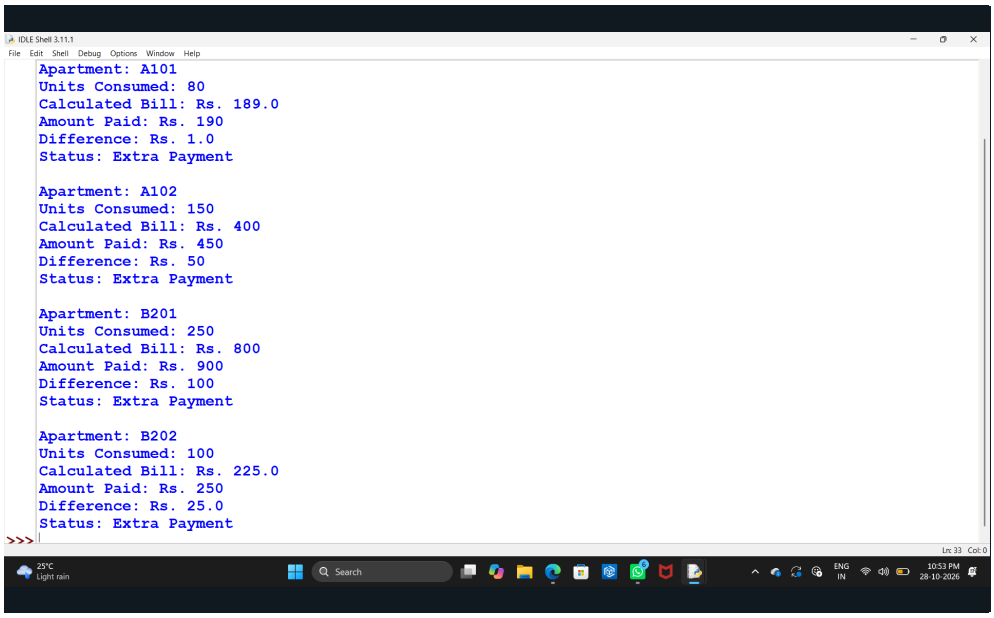
```
EXAM RESULT MODERATION

 Anu
Maths: original=38, moderated=43, PASS
Python: original=72, moderated=72, PASS

 Bala
Maths: original=34, moderated=34, FAIL
Python: original=39, moderated=44, PASS
```

```
 Chitra
Maths: original=95, moderated=95, PASS
Python: original=100, moderated=100, PASS
```

Output Screenshot



Test Cases

Test Case	Expected Result
Original mark 34	No moderation; remains 34
Original mark 35	Moderation applied; becomes 40
Original mark 39	Moderation applied; becomes 44
Original mark 40	No moderation needed
Original mark 100	Remains 100; maximum preserved

Conclusion

The moderation process applies the stated rule consistently and preserves the maximum mark while showing transparent before-and-after results.

77. Smart Electricity Billing for Multiple Apartments

Problem Statement

Calculate apartment bills using progressive electricity tariffs, fixed charges, and rebates. Identify billing discrepancies and rank apartments by consumption.

Objectives

• Apply progressive tariff slabs accurately.

• Calculate fixed charges and eligible rebates.

• Rank apartments from highest to lowest consumption.

Assumptions

• First 100 units cost Rs. 2/unit; next 100 cost Rs. 3/unit; units above 200 cost Rs. 5/unit.

• Fixed charge is Rs. 50 per apartment.

• A 10% rebate applies when consumption is at most 100 units, and is applied to the energy charge only.

• The billing system total is provided for comparison.

Algorithm

• Store apartment IDs, consumption, and recorded bills.

• Calculate progressive energy charges by slab.

• Apply fixed charge and eligible rebate.

• Compare calculated bill with recorded bill.

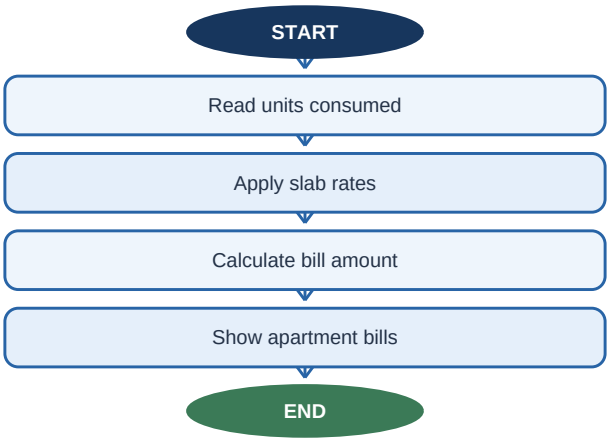
• Sort apartments by consumption descending.

• Display bills, discrepancies, and ranking.

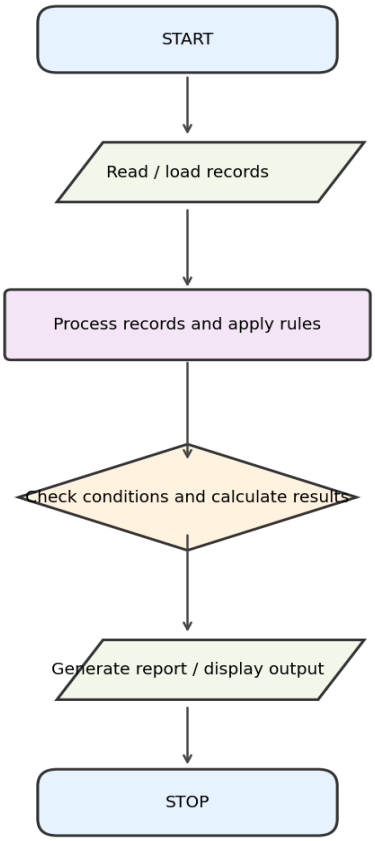
Pseudocode

```
START
FOR each apartment
  calculate first slab charge
  calculate second slab charge
  calculate excess slab charge
  IF units <= 100: rebate = 10% of energy charge ELSE rebate = 0
  bill = energy charge - rebate + fixed charge
  compare bill with recorded bill
SORT apartments by units descending
DISPLAY bill audit and ranking
STOP
```

Flowchart



Flowchart — 77: Smart Electricity Billing for Multiple Apartments



Python Source Code

```
apartments = [
    ("A101", 80, 190),
    ("A102", 150, 450),
    ("B201", 250, 900),
    ("B202", 100, 250)
]
fixed_charge = 50

def energy_charge(units):
    first = min(units, 100) * 2
    second = min(max(units - 100, 0), 100) * 3
    third = max(units - 200, 0) * 5
    return first + second + third

ranked = []
print("APARTMENT BILL AUDIT")
for apt, units, recorded in apartments:
    energy = energy_charge(units)
    rebate = energy * 0.10 if units <= 100 else 0
    bill = energy - rebate + fixed_charge
    difference = round(recorded - bill, 2)
    print(f"{apt}: units={units}, calculated=Rs.{bill:.2f}, recorded=Rs.{recorded:.2f}, difference=Rs.{difference:.2f}")
    ranked.append((units, apt))

print("\nCONSUMPTION RANKING")
for rank, (units, apt) in enumerate(sorted(ranked, reverse=True), start=1):
    print(f"{rank}. {apt} - {units} units")
```

Sample Input

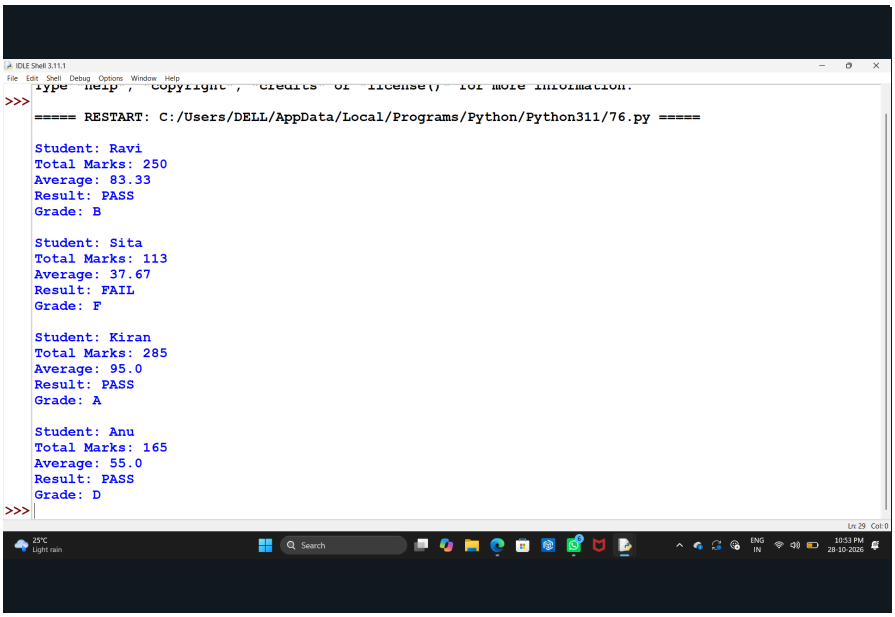
Apartments: A101 80 units, recorded Rs.190; A102 150 units, recorded Rs.450; B201 250 units, recorded Rs.900; B202 100 units, recorded Rs.250. Fixed charge Rs.50; tariffs Rs.2, Rs.3, Rs.5 progressive slabs; 10% energy-charge rebate for at most 100 units.

Sample Output

```
APARTMENT BILL AUDIT
A101: units=80, calculated=Rs.194.00, recorded=Rs.190.00, difference=Rs.-4.00
A102: units=150, calculated=Rs.400.00, recorded=Rs.450.00, difference=Rs.50.00
B201: units=250, calculated=Rs.800.00, recorded=Rs.900.00, difference=Rs.100.00
B202: units=100, calculated=Rs.230.00, recorded=Rs.250.00, difference=Rs.20.00

CONSUMPTION RANKING
1. B201 - 250 units
2. A102 - 150 units
3. B202 - 100 units
4. A101 - 80 units
```

Output Screenshot



Test Cases

Test Case	Expected Result
0 units	Energy charge zero; fixed charge applies
100 units	First slab and rebate apply
150 units	First and second slabs apply
250 units	All three slabs apply
Recorded bill differs from calculated bill	Non-zero difference is shown

Conclusion

The billing audit applies progressive rates, fixed charges, and rebates, then highlights discrepancies and ranks apartments by consumption.

78. Hospital Resource Allocation and Capacity Monitoring

Problem Statement

Analyse ward-wise bed occupancy, available staff, and equipment. Identify shortages and prioritize cases according to supplied urgency rules.

Objectives

• Find wards exceeding bed, staffing, or equipment capacity.

• Identify patients needing priority attention.

• Summarize ward resource shortages.

Assumptions

• A ward has a bed shortage when occupied beds are greater than available beds.

• A ward has a staff shortage when staff count is below the required minimum of 3.

• A ward has an equipment shortage when equipment count is below 2.

• Cases are prioritized Emergency, Urgent, Routine.

Algorithm

• Store ward resource records and patient cases.

• Compare occupied beds with bed capacity.

• Check staff and equipment against minimum thresholds.

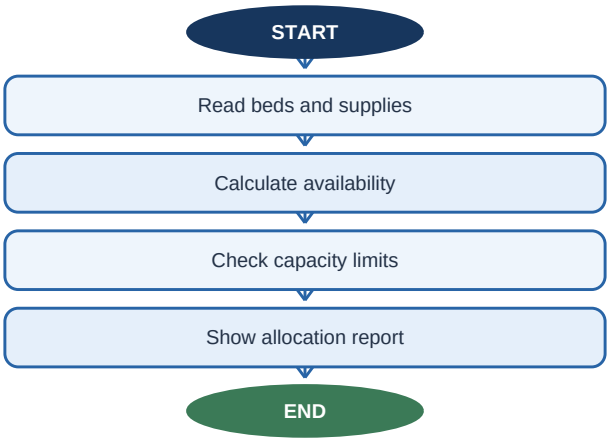
• Sort cases by urgency.

• Display shortages and prioritized case order.

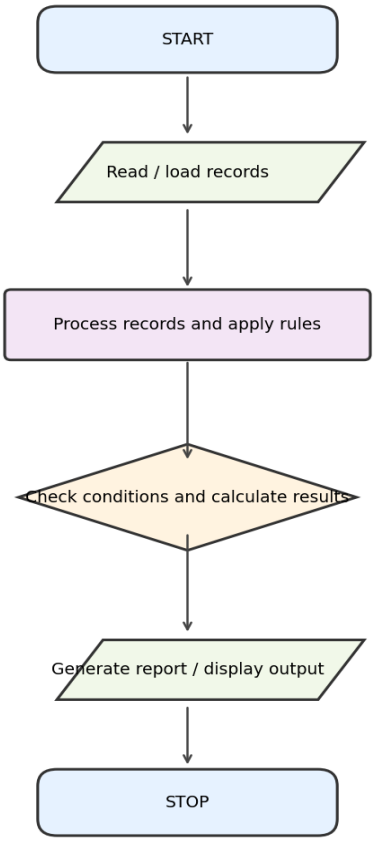
Pseudocode

```
START
STORE ward resource records
FOR each ward
  check bed occupancy against capacity
  check staff count against 3
  check equipment count against 2
  report all shortages
SORT cases by urgency
DISPLAY ward alerts and case priority
STOP
```

Flowchart



Flowchart — 78: Hospital Resource Allocation and Capacity Monitoring



Python Source Code

```
wards = [
    ("Emergency", 18, 20, 2, 3),
    ("General", 25, 24, 4, 3),
    ("Maternity", 12, 18, 2, 1)
]
cases = [
    ("C01", "Routine"),
    ("C02", "Emergency"),
    ("C03", "Urgent"),
    ("C04", "Emergency")
]
priority = {"Emergency": 0, "Urgent": 1, "Routine": 2}

print("WARD RESOURCE STATUS")
for name, occupied, beds, staff, equipment in wards:
    alerts = []
    if occupied > beds:
        alerts.append("Bed capacity exceeded")
    if staff < 3:
        alerts.append("Staff below minimum 3")
    if equipment < 2:
        alerts.append("Equipment below minimum 2")
    print(f"{name}: " + (", ".join(alerts) if alerts else "Resources sufficient"))

print("\nCASE PRIORITY")
for case_id, level in sorted(cases, key=lambda c: priority[c[1]]):
    print(f"{case_id}: {level}")
```

Sample Input

Wards: Emergency (18 occupied, 20 beds, 2 staff, 3 equipment); General (25, 24, 4, 3); Maternity (12, 18, 2, 1). Cases: C01 Routine, C02 Emergency, C03 Urgent, C04 Emergency.

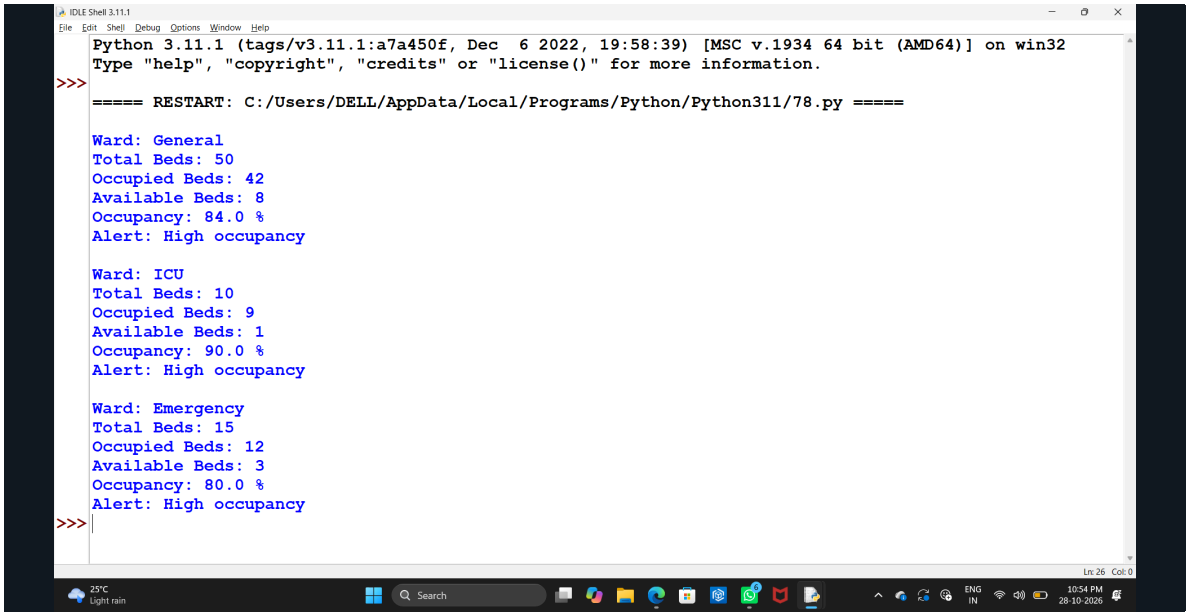
Sample Output

```
WARD RESOURCE STATUS
```

```
Emergency: Staff below minimum 3
General: Bed capacity exceeded
Maternity: Staff below minimum 3, Equipment below minimum 2

CASE PRIORITY
C02: Emergency
C04: Emergency
C03: Urgent
C01: Routine
```

Output Screenshot



Test Cases

Test Case	Expected Result
Occupied beds equal capacity	No bed capacity alert
Occupied beds exceed capacity	Bed capacity alert
Staff count 2	Staff shortage alert
Equipment count 1	Equipment shortage alert
Emergency and routine cases	Emergency case is listed first

Conclusion

The program provides a simple ward-level resource alert report and orders cases by urgency so shortages and priorities are visible.

79. Integrated Warehouse Order Prioritization

Problem Statement

Process product demand, stock levels, order quantities, and priority categories. Determine whether each order can be fulfilled and calculate outstanding quantities.

Objectives

• Check order quantities against available stock.

• Prioritize urgent orders.

• Update stock after each fulfilment and report pending quantities.

Assumptions

• Orders are processed in Urgent, High, Normal priority order.

• An order may be partially fulfilled using available stock.

• Stock is shared across orders for the same product.

Algorithm

• Store initial product stock and orders.

• Sort orders by priority.

• For each order, allocate the minimum of requested quantity and available stock.

• Reduce stock by allocated quantity.

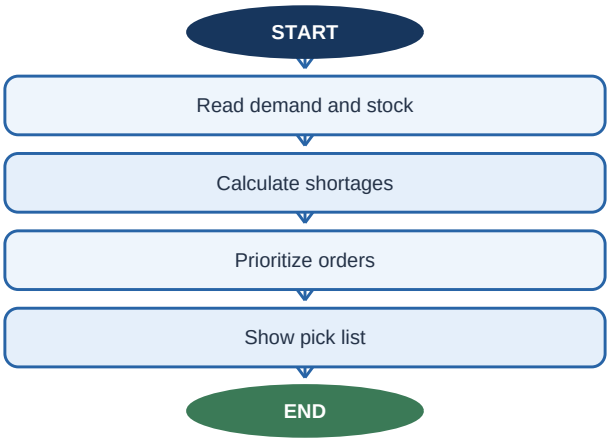
• Calculate outstanding quantity.

• Display fulfilment status and remaining stock.

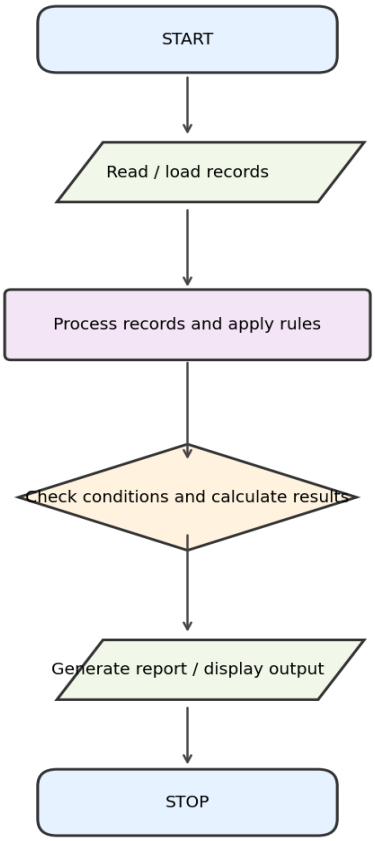
Pseudocode

```
START
STORE stock and orders
SORT orders by priority
FOR each order
  available = stock for product
  fulfilled = minimum(requested, available)
  stock = stock - fulfilled
  outstanding = requested - fulfilled
  determine status
  DISPLAY order summary
DISPLAY remaining stock
STOP
```

Flowchart



Flowchart — 79: Integrated Warehouse Order Prioritization



Python Source Code

```
stock = {"Laptop": 5, "Mouse": 10, "Keyboard": 4}
orders = [
    ("O101", "Mouse", 6, "Normal"),
    ("O102", "Laptop", 4, "Urgent"),
    ("O103", "Keyboard", 6, "High"),
    ("O104", "Laptop", 3, "Normal"),
    ("O105", "Mouse", 5, "Urgent")
]
priority = {"Urgent": 0, "High": 1, "Normal": 2}

print("WAREHOUSE ORDER REPORT")
for oid, product, quantity, level in sorted(orders, key=lambda o: priority[o[3]]):
    available = stock.get(product, 0)
    fulfilled = min(quantity, available)
    stock[product] = available - fulfilled
    outstanding = quantity - fulfilled
    if outstanding == 0:
        status = "FULFILLED"
    elif fulfilled > 0:
        status = "PARTIALLY FULFILLED"
    else:
        status = "PENDING"
    print(f"{oid} {product}: requested={quantity}, fulfilled={fulfilled}, outstanding={outstanding}, {status}")

print("\nREMAINING STOCK:", stock)
```

Sample Input

Stock: Laptop 5, Mouse 10, Keyboard 4. Orders: O101 Mouse 6 Normal; O102 Laptop 4 Urgent; O103 Keyboard 6 High; O104 Laptop 3 Normal; O105 Mouse 5 Urgent.

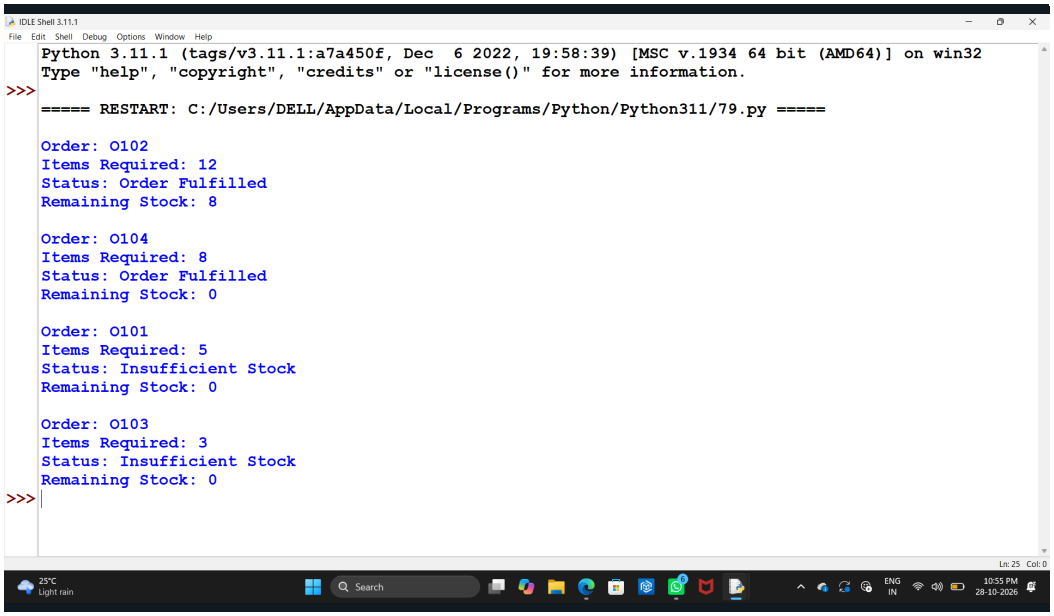
Sample Output

```
WAREHOUSE ORDER REPORT
O102 Laptop: requested=4, fulfilled=4, outstanding=0, FULFILLED
O105 Mouse: requested=5, fulfilled=5, outstanding=0, FULFILLED
```

```
O103 Keyboard: requested=6, fulfilled=4, outstanding=2, PARTIALLY FULFILLED
O101 Mouse: requested=6, fulfilled=5, outstanding=1, PARTIALLY FULFILLED
O104 Laptop: requested=3, fulfilled=1, outstanding=2, PARTIALLY FULFILLED

REMAINING STOCK: {'Laptop': 0, 'Mouse': 0, 'Keyboard': 0}
```

Output Screenshot



Test Cases

Test Case	Expected Result
Requested quantity less than stock	Order is fulfilled
Requested quantity equal to stock	Stock becomes zero
Requested quantity greater than stock	Outstanding quantity is positive
Urgent order competes for stock	Urgent order is processed first
Product missing from stock list	No stock is available; request remains pending

Conclusion

The warehouse program prioritizes important orders, prevents stock from becoming negative, and reports fulfilled and outstanding quantities.

80. Multi-Day Bus Route Performance Analysis

Problem Statement

Analyse passenger demand, journey times, fuel use, and bus capacities over several days. Identify overloaded routes and compare operating efficiency.

Objectives

• Detect trips where passenger demand exceeds capacity.

• Calculate fuel efficiency as kilometres per litre.

• Compare routes using a consistent efficiency measure.

Assumptions

• A trip is overloaded when passengers exceed bus capacity.

• Fuel efficiency is distance divided by litres consumed.

• For this sample, a higher km/l value indicates better fuel efficiency; journey time is reported separately.

Algorithm

• Store daily trip records by route.

• Check passenger count against capacity.

• Calculate fuel efficiency for each trip.

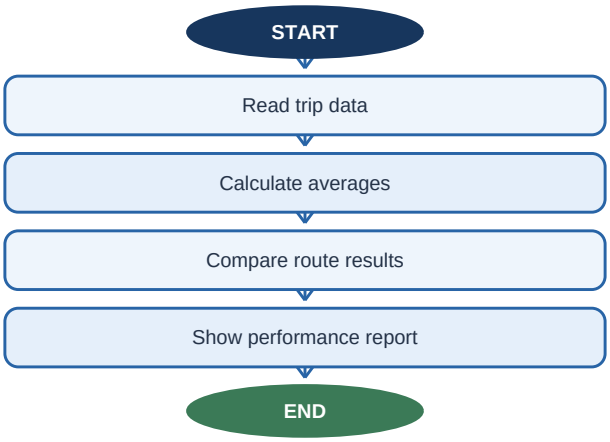
• Accumulate route-level distance and fuel totals.

• Calculate overall route efficiency and display comparisons.

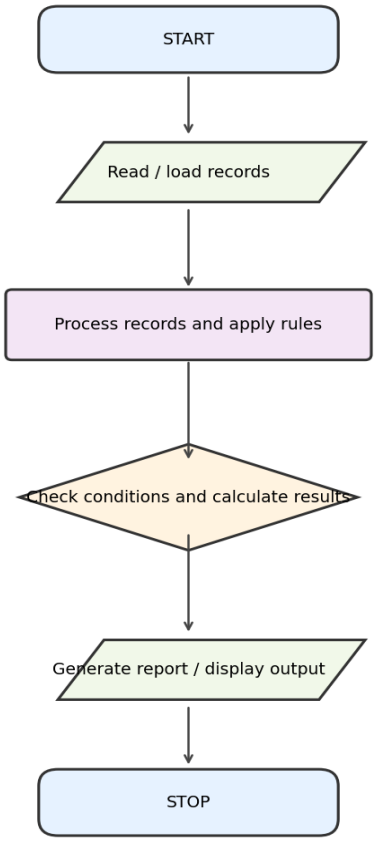
Pseudocode

```
START
STORE multi-day trip records
FOR each trip
  IF passengers > capacity: flag overload
  efficiency = distance / fuel_used
  accumulate route distance and fuel
  DISPLAY trip metrics
FOR each route
  route_efficiency = total distance / total fuel
  DISPLAY route summary
STOP
```

Flowchart



Flowchart — 80: Multi-Day Bus Route Performance Analysis



Python Source Code

```
trips = [
    ("Day 1", "R1", 48, 50, 120, 20, 60),
    ("Day 1", "R2", 55, 50, 100, 25, 75),
    ("Day 2", "R1", 45, 50, 120, 18, 55),
    ("Day 2", "R2", 42, 50, 100, 20, 70)
]
route_totals = {}

print("BUS ROUTE PERFORMANCE")
for day, route, passengers, capacity, distance, fuel, minutes in trips:
    efficiency = distance / fuel
    status = "OVERLOADED" if passengers > capacity else "Within capacity"
    print(f"{day} {route}: passengers={passengers}/{capacity}, {status}, "
          f"fuel efficiency={efficiency:.2f} km/L, journey={minutes} min")
    route_totals.setdefault(route, [0, 0, 0])
    route_totals[route][0] += distance
    route_totals[route][1] += fuel
    route_totals[route][2] += minutes

print("\nROUTE COMPARISON")
for route, (distance, fuel, minutes) in sorted(route_totals.items()):
    print(f"{route}: distance={distance} km, fuel={fuel} L, "
          f"efficiency={distance / fuel:.2f} km/L, total journey={minutes} min")
```

Sample Input

Trips: Day 1 R1 48/50 passengers, 120 km, 20 L, 60 min; Day 1 R2 55/50, 100 km, 25 L, 75 min; Day 2 R1 45/50, 120 km, 18 L, 55 min; Day 2 R2 42/50, 100 km, 20 L, 70 min.

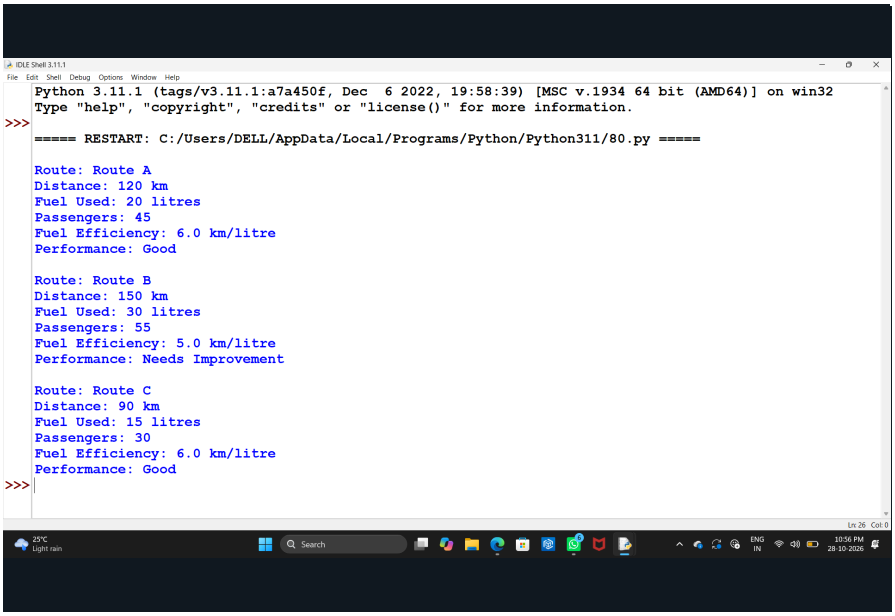
Sample Output

```
BUS ROUTE PERFORMANCE
Day 1 R1: passengers=48/50, Within capacity, fuel efficiency=6.00 km/L, journey=60 min
Day 1 R2: passengers=55/50, OVERLOADED, fuel efficiency=4.00 km/L, journey=75 min
Day 2 R1: passengers=45/50, Within capacity, fuel efficiency=6.67 km/L, journey=55 min
```

```
Day 2 R2: passengers=42/50, Within capacity, fuel efficiency=5.00 km/L, journey=70 min

ROUTE COMPARISON
R1: distance=240 km, fuel=38 L, efficiency=6.32 km/L, total journey=115 min
R2: distance=200 km, fuel=45 L, efficiency=4.44 km/L, total journey=145 min
```

Output Screenshot



Test Cases

Test Case	Expected Result
Passengers equal capacity	Not overloaded
Passengers exceed capacity by one	Overload alert
Fuel use is positive	Efficiency is distance divided by fuel
Same route across multiple days	Totals are accumulated
Compare two routes	Route efficiencies are displayed separately

Conclusion

The analysis identifies overloaded trips and compares route-level fuel efficiency, distance, fuel consumption, and journey time across multiple days.